BIRZEIT UNIVERSITY

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS4370, Computer Architecture

Project No. 2

Design and Verification of a Simple Pipelined RISC Processor in Verilog

---

Prepared by:

Sondos Hammad	**ID:** 1230521	**Section:** 3
Sarah Akkawi	**ID:** 1230448	**Section:** 2
Jana Snaf	**ID:** 1230446	**Section:** 2

Instructor: Dr.Ahmad Afaneh

Date: 6/1/2025

Abstract

This project focuses on the design, implementation, and verification of a 32-bit pipelined RISC processor using Verilog. The processor follows a classic five-stage pipeline instruction fetch, decode, execute, memory access, and write-back, and incorporates a general-purpose register file with separate instruction and data memories to streamline memory operations and improve performance. The supported ISA uses a fixed 32-bit instruction format and includes arithmetic and logical operations (R-type), immediate instructions (I-type), load/store memory operations, and control-flow instructions (J-type) such as branches and jumps. A key feature of the ISA is predicated execution, where each instruction includes a predicate register (Rp) that determines whether the instruction takes effect. The work covers designing the processor architecture and datapath, implementing the pipeline and instruction set in Verilog, and developing the control logic required for correct operation. Functionality was verified entirely in Verilog using a comprehensive testbench that runs multiple test programs, with simulation waveforms analyzed to confirm correct instruction execution and control behavior.

Table of Contents

Table of figures

List of Tables

1. Theory

The processor designed in this project is a simple 32-bit RISC processor with a consistent 32-bit instruction format. It follows the RISC design approach by using a small, efficient instruction set and separate instruction and data memories that are word-addressable. The processor includes 32 general-purpose registers (R0 to R31), where R0 is hardwired to zero, R30 serves as the Program Counter (PC), and R31 is used as the return address register. A key feature of this design is predicated execution, where each instruction contains a predicate register (Rp) that determines whether the instruction's effects are applied.

1.1 Instruction Set Architectur(ISA) :

An Instruction Set Architecture (ISA) is part of the abstract model of a computer that defines how the CPU is controlled by the software. The ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done.[1]

In this project, we implement a custom 32-bit predicated RISC-style ISA in Verilog, where the instruction and word size are 32 bits and the processor contains 32 general-purpose registers (R0–R31), with R0 hardwired to zero, R30 used as the program counter (PC), and R31 used as the return address register.

The processor uses separate instruction and data memories (word-addressable), and it supports predicated execution through a predicate register Rp

if Reg [Rp] = 0 → the instruction does not execute

if Reg [Rp] ≠ 0 → it executes (Rp also used to allow unconditional execution).

The ISA uses three instruction formats (R-type, I-type, and J-type) with fixed fields for opcode and register specifiers, plus an immediate/offset field depending on the type, and it includes arithmetic/logical operations, immediate variants, load/store instructions, and control-flow instructions

Instruction Format:

1. Register type (R-type):

$Opcode^5$	Rp^5	Rd^5	Rs^5	Rt^5	$Unused^7$

- **Opcode (5 bits):** Specifies the operation to be performed (ADD, SUB).
- **Rp (5 bits):** Predicate register, it selects which register controls whether the instruction is executed or skipped.

  If Reg[Rp] = 0 → the instruction is suppressed (no register write, no memory write, no control-flow change).

  If Reg[Rp] ≠ 0 → the instruction executes normally.
- **Rd (5 bits):** Destination register, the register that receives the result.
- **Rs (5 bits):** First source register.
- **Rt (5 bits):** Second source register.
- **Unused (7 bits):** Unused in R-type instructions, these bits are typically set to 0 and are included only to keep the instruction length at 32 bits

2. Immediate type (I-type):

$Opcode^5$	Rp^5	Rd^5	Rs^5	$Immediate^{12}$

- **Opcode (5 bits):** Specifies the operation to be performed (ADD, SUB).
- **Rp (5 bits):** Predicate register,it selects which register controls whether the instruction is executed or skipped.(as detailed in the R-type)
- **Rd (5 bits):** Destination register for the result (for ALU-immediate and LW).

  For SW, Rd is the source register whose value is stored to memory.
- **Rs (5 bits):** Source register, Used as the first ALU operand.
- **Immediate (12 bits):** Offset value used by the instruction.

  Sign-extended for arithmetic and address offsets (ADDI, LW, SW).

  Zero-extended for logical immediate (ANDI, ORI, NORI).

3. Jump type (J-type):

Opcode[5]	Rp[5]	Offset[22]

- **Opcode (5 bits):** specifies the control-flow instruction (J, CALL).
- **Rp (5 bits):** Predicate register, The CPU checks Reg[Rp] to decide whether the jump takes effect:

  If Reg[Rp] = 0 → the jump is suppressed (PC is not changed by the jump).

  If Reg[Rp] ≠ 0 → the jump executes normally.
- **Offset (22 bits):** Signed offset used to compute the jump target relative to the current PC.

  The target address is typically computed as: **PC_next = PC+ sign_extend(Offset)** (word addressing)

The processor supports a selected subset of instructions, grouped into arithmetic, logical, memory-access, and control-flow operations. These instructions form the core of typical programs, and the design executes them efficiently using a five-stage pipelined datapath with coordinated control and timing, while supporting predicated execution through the Rp field to conditionally enable instruction effects.

1.2 Instruction Categories :

i) Arithmetic Instructions:

Arithmetic instructions perform basic integer operations either between two registers or between a register and an immediate constant.

Two categories of arithmetic instructions are typically used to perform integer addition and subtraction.

- **Signed arithmetic:** 32-bit operands are interpreted as **two's complement** values; addition/subtraction can cause **overflow** if the result exceeds the signed 32-bit range. [2]
- **Unsigned arithmetic:** the 32 bit numbers are considered to be in standard binary representation. Executing the instruction will never generate an overflow error even if there is an actual overflow (the result cannot be represented with 32 bits).[2]

In this project, we implement and use the following arithmetic instructions to perform basic integer computations within our programs and test cases: **ADD**, **ADDI**, and **SUB.**

- **ADD**: Adds two register values and stores the result in a destination register.
- **ADDI**: Adds a register value and an immediate constant and stores the result in a destination register.
- **SUB**: Subtracts the second register from the first and stores the result.

These instructions are essential for performing computations such as counters, sums, and index calculations.

ii) Logical Instructions:

Logical instructions perform bitwise operations on operands

- **OR:** Performs a bitwise OR between two register operands and writes the result to the destination register.
- **ORI:** Performs a bitwise OR between a register operand and an immediate value and writes the result to the destination register.
- **AND:** Performs a bitwise AND between two register operands and writes the result to the destination register.
- **ANDI:** Performs a bitwise AND between a register operand and an immediate value and writes the result to the destination register.
- **NOR:** Performs a bitwise NOR (~(A | B)) between two register operands and writes the result to the destination register.
- **NORI:** Performs a bitwise NOR (~(A | Imm)) between a register operand and an immediate value and writes the result to the destination register.

Logical operations are often used in mask operations, flag setting, or conditional bit manipulation.

iii) Memory Access Instructions:

Memory access instructions are CPU operations that transfer data between the register file and main memory. They are used to read data from memory into a register (load) or write data from a register back to memory (store).

- Load (LW) to fetch data from memory to registers.
- Store (SW) to write data from registers back to memory.

These instructions are essential for accessing data stored outside the processor, such as arrays and global variables, since most computations are performed in registers and therefore require explicit load/store operations to interact with memory.

iv) Control Flow Instructions:

Control flow instructions modify the normal sequential execution of the program by updating the Program Counter (PC) to a new target address.allowing for loops, conditionals, and function calls.

- **JR (Jump Register)**: Jumps to an address stored in a register.
- **J (Unconditional Jump)**: Jumps to a new instruction address unconditionally.
- **CLL (Call)**: Calls a subroutine by jumping to an address and saving the return address in R31, allowing the program to return later using JR.

All three instructions are predicated using Rp: they only take effect if $Reg[Rp] \neq 0$.

If $Reg[Rp] = 0$, the instruction is suppressed (no jump/call/jr effect).

1.3 Datapath Design Approaches

This section explains the main CPU datapath designs and highlights how each one executes instructions, along with their key advantages and limitations.

i) Single cycle :

A single-cycle datapath completes every instruction in exactly one clock cycle, making it the simplest processor design. In this architecture, all stages (instruction fetch, decode, execute, memory access (if needed), and write-back) are performed within the same cycle. This greatly simplifies the control logic. However, it reduces efficiency because the clock period must be long enough to accommodate the slowest instruction, so even simple operations (like arithmetic instructions) are forced to run at the same long cycle time required by more complex instructions (such as loads and memory operations).[3]

ii) Multi Cycle:

A multi-cycle CPU breaks each instruction into a sequence of smaller stages executed over multiple clock cycles, such as IF, ID, EX, MEM, and WB, with each cycle performing only part of the instruction. This allows a shorter clock period than a single-cycle design and enables hardware reuse (ALU can be used in different cycles), which reduces hardware area and improves utilization. Because different instructions require different stages, they take different numbers of cycles—for example, an ADD may complete in fewer cycles than a load—making the design more time-efficient than single-cycle. However, since instructions span multiple cycles, the CPI is greater than 1, and the processor requires more complex control logic, usually supported by extra registers that hold intermediate values between cycles

iii) Pipeline:

A pipelined CPU is a design technique that improves performance by dividing instruction execution into five stages—Fetch, Decode, Execute, Memory, and Writeback—and allowing multiple instructions to be processed at the same time in different stages, similar to an assembly line. In the IF stage, the processor fetches the instruction and updates the PC; in ID, it decodes the instruction, reads registers, and produces control signals; in EX, it performs the ALU operation or calculates an effective address; in MEM, it accesses data memory for loads and stores; and in WB,

it writes the final result back to the register file. While pipelining increases throughput by overlapping these tasks, it also introduces challenges such as data, control, and structural hazards, which must be handled using techniques like forwarding, and stalling.

- ## Pipeline Hazards:

1. Structural Hazards

A structural hazard, occurs when two or more instructions require access to the same hardware resource simultaneously, and the hardware cannot support the required parallel access. [4]

Solution to Structural Hazards:

To reduce stalls, the CPU can use separate memories: one for instructions and one for data. This way, the processor can fetch an instruction and access data at the same time, so the pipeline stages don't compete for the same memory and fewer stalls happen.

2. Control Hazards (Branch hazard)

A control hazard happens when a CPU can't tell which instructions it needs to execute next.it occurs when a branch or jump changes the normal flow of execution. Because the next instruction address depends on the branch outcome, the pipeline may fetch and partially execute the wrong instructions before the decision is known. When the branch is resolved, those incorrect instructions are typically discarded, causing a performance penalty that depends on the pipeline depth and where the branch decision is made.

Solution for Control Hazards:

Branch Prediction: Hardware predicts the outcome of a branch to continue fetching before the condition resolves.

Delayed branching: is a technique where the compiler rearranges instructions so that the instruction immediately after a branch (the **delay slot**) always executes, regardless of whether the branch is taken or not. By filling this delay slot with a useful, independent instruction instead of a NOP, the pipeline can avoid wasting a cycle and reduce the performance penalty of control hazards.

3. Data Hazards

A data hazard occurs when instructions exhibit data dependencies such that one instruction depends on the result of a previous instruction that has not yet completed in the pipeline. [4]

Solution for Data Hazards:

Forwarding (bypassing): The CPU routes the needed result directly from a later pipeline stage to the instruction that requires it, instead of waiting for it to be written back to the register file. [5]

Stalling (inserting bubbles/NOPs): The dependent instruction is delayed for one or more cycles, effectively inserting one or more NOP cycles until the required data becomes available. [5]

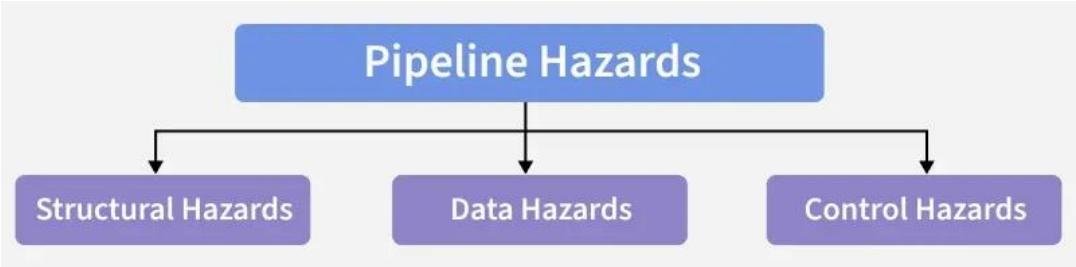


Figure 1: Data hazard types[4]

Control unit, inst mem, r file,reg between stages, alu.

3.1 Instruction Encoding Table

Instruction	Type	Meaning (RTN)	Opcode
ADD Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] + Reg[Rt]	0
SUB Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] - Reg[Rt]	1
OR Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] | Reg[Rt]	2
NOR Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = ~(Reg[Rs] | Reg[Rt])	3
AND Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] & Reg[Rt]	4

ADDI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] + Imm	5
ORI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] | Imm	6
NORI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = ~(Reg[Rs] | Imm)	7
ANDI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Reg[Rs] & Imm	8
LW Rd, Imm(Rs), Rp	I-Type	If (Reg[Rp] ≠ 0) Reg[Rd] = Mem(Reg[Rs] + Imm)	9
SW Rd, Imm(Rs), Rp	I-Type	If (Reg[Rp] ≠ 0) Mem(Reg[Rs] + Imm) = Reg[Rd]	10
J Label, Rp	J-Type	If (Reg[Rp] ≠ 0) jump to the target address	11
CALL Label, Rp	J-Type	If (Reg[Rp] ≠ 0) jump to the function; store the return address in register R31	12
JR, Rs, Rp	R-Type	If (Reg[Rp] ≠ 0) jump to the address in register Rs	13

Table 1: Instruction Encoding Table

Clarification: In this ISA, each instruction includes a predicate register field (Rp) that enables predicated execution, meaning the instruction only takes effect when a condition is satisfied. During the decode stage, the processor reads the value of the register selected by Rp and generates an execution enable signal (Pred). If Reg[Rp] ≠ 0, the instruction executes normally; otherwise, it is suppressed and behaves like a NOP (no register write, no memory write, and no control-flow update). To support unconditional execution, the ISA treats Rp = R0 as a special case, forcing Pred = 1 so the instruction executes regardless of register values. This mechanism provides conditional behavior without requiring separate branch instructions and is applied consistently across all instruction types.

Instruction	Stage	RTL
ADD Rd, Rs, Rt, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1

	ID (Decode)	Opcode = inst[31:27]
		Rp = inst[26:22]
		Rd = inst[21:17]
		Rs = inst[16:12]
		Rt = inst[11:7]
		Pred = (Rp == 0) || (Reg[Rp] != 0)
		
		//if Rp==R0==0 unconditional ,pred=1 //if (Reg[Rp] != 0) , pred=1
		ALUOperand1 = Reg[Rs]
		ALUOperand2 = Reg[Rt]
		PC_next = PC

	EX (Execute)	if Pred: ALUResult = ALUOperand1 + ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next

SUB Rd, Rs, Rt, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Rt = inst[11:7] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ALUOperand2 = Reg[Rt] PC_next = PC

	EX (Execute)	if Pred: ALUResult = ALUOperand1 - ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next
OR Rd, Rs, Rt, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Rt = inst[11:7] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ALUOperand2 = Reg[Rt] PC_next = PC
	EX (Execute)	if Pred: ALUResult = ALUOperand1 | ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next

NOR Rd, Rs, Rt, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Rt = inst[11:7] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs]

		ALUOperand2 = Reg[Rt] PC_next = PC
	EX (Execute)	if Pred: ALUResult = ~(ALUOperand1 | ALUOperand2)
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next
AND Rd, Rs, Rt, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Rt = inst[11:7] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ALUOperand2 = Reg[Rt] PC_next = PC
	EX (Execute)	if Pred: ALUResult = ALUOperand1 & ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next

ADDI Rd, Rs, Imm, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Imm = inst[11:0]

		Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ImmExt = SignExtend(Imm) ALUOperand2 = ImmExt PC_next = PC
	EX (Execute)	if Pred: ALUResult = ALUOperand1 + ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next
ORI Rd, Rs, Imm, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Imm = inst[11:0] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ImmExt = ZeroExtend(Imm) ALUOperand2 = ImmExt PC_next = PC
	EX (Execute)	if Pred: ALUResult = ALUOperand1 | ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next
NORI Rd, Rs, Imm, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17]

		Rs = inst[16:12] Imm = inst[11:0] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ImmExt = ZeroExtend(Imm) ALUOperand2 = ImmExt PC_next = PC
	EX (Execute)	if Pred: ALUResult = ~(ALUOperand1 | ALUOperand2)
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next
ANDI Rd, Rs, Imm, Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Imm = inst[11:0] Pred = (Rp == 0) || (Reg[Rp] != 0) ALUOperand1 = Reg[Rs] ImmExt = ZeroExtend(Imm) ALUOperand2 = ImmExt PC_next = PC
	EX (Execute)	if Pred: ALUResult = ALUOperand1 & ALUOperand2
	MEM (Memory)	Not required
	WB (Writeback)	if Pred: Reg[Rd] = ALUResult PC = PC_next

LW Rd, Imm(Rs), Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Imm = inst[11:0] Pred = (Rp == 0) || (Reg[Rp] != 0) Base = Reg[Rs] ImmExt = SignExtend(Imm) PC_next = PC
	EX (Execute)	if Pred: Addr = Base + ImmExt
	MEM (Memory)	if Pred: MemData = dataMem[Addr]
	WB (Writeback)	if Pred: Reg[Rd] = MemData PC = PC_next
SW Rd, Imm(Rs), Rp	IF (Fetch)	inst = instMem[PC] PC = PC + 1
	ID (Decode)	Opcode = inst[31:27] Rp = inst[26:22] Rd = inst[21:17] Rs = inst[16:12] Imm = inst[11:0] Pred = (Rp == 0) || (Reg[Rp] != 0) Base = Reg[Rs] StoreData = Reg[Rd] ImmExt = SignExtend(Imm) PC_next = PC
	EX (Execute)	if Pred: Addr = Base + ImmExt
	MEM (Memory)	if Pred: dataMem[Addr] = StoreData

	WB (Writeback)	Not required PC = PC_next

J Label, Rp	IF (Fetch)	PC_old = PC
		inst = instMem[PC=old]
		//to bring inst from the current pc
		NPC = PC + 1 //next pc

	ID (Decode)	Opcode = inst[31:27]
		Rp = inst[26:22]
		Offset = inst[21:0]
		Pred = (Reg[Rp] != 0)
		OffExt = SignExtend(Offset)
		Target = PC_old + OffExt
		if Pred:
		PC_next = Target
		else
		Pc_next=NPC

	EX (Execute)	Not required
	MEM (Memory)	Not required
	WB (Writeback)	Not required PC = PC_next
CALL Label, Rp	IF (Fetch)	inst = instMem[PC] NPC = PC + 1

	ID (Decode)	Opcode = inst[31:27]
		Rp = inst[26:22]
		Offset = inst[21:0]
		Pred = (Rp == 0) || (Reg[Rp] != 0)
		OffExt = SignExtend(Offset)
		Target = PC + OffExt
		RetAddr = PC
		if Pred:
		PC_next = Target
		Else
		PC_next = NPC

	EX (Execute)	Not required
	MEM (Memory)	Not required
	WB (Writeback)	Reg[R31] = PC_next

JR Rs, Rp	IF (Fetch)	inst = instMem[PC] NPC = PC + 1

	ID (Decode)	Opcode = inst[31:27]
		Rp = inst[26:22]
		Rs = inst[16:12]
		Pred = (Rp == 0) || (Reg[Rp] != 0)
		Target = Reg[Rs]
		if Pred:
		PC_next = Target
		
		Else
		PC_next=NPC

	EX (Execute)	Not required
	MEM (Memory)	Not required
	WB (Writeback)	Not required

2. Procedure

2.1 Instruction Fetch (IF)

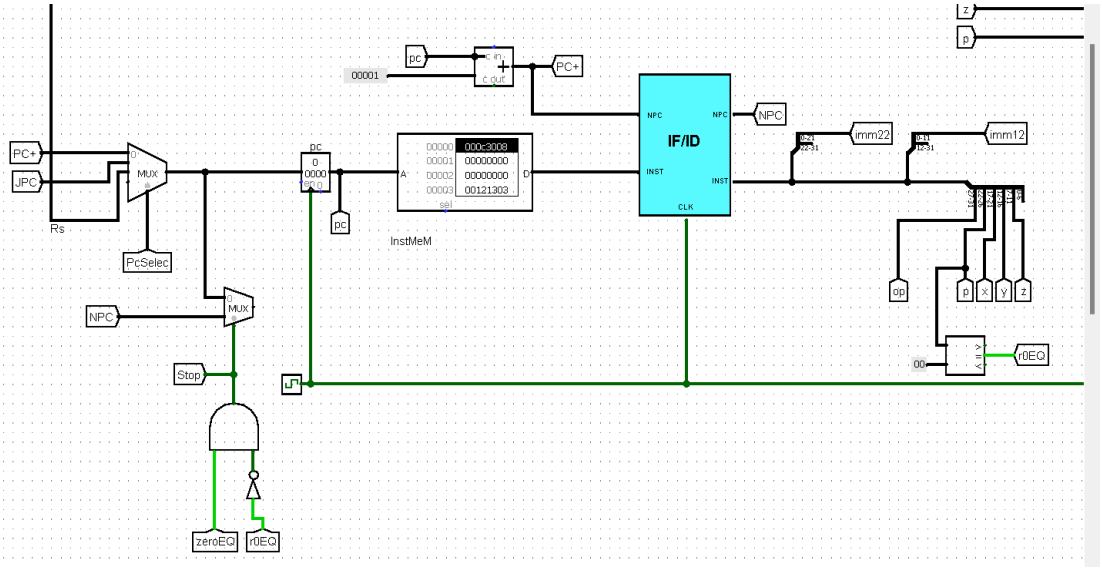


Figure 2: IF

General explanation of the Instruction Fetch (IF) stage:

The Instruction Fetch (IF) stage is the first stage in a pipelined CPU. Its main job is to retrieve the next instruction from instruction memory using the Program Counter (PC). During each clock cycle, the PC provides an address to the instruction memory, which returns the corresponding instruction. At the same time, the fetch stage computes the next sequential address (PC + 1, because the machine is word-addressable). Finally, the fetch stage sends both the fetched instruction (INST) and the next PC (NPC) to the next pipeline stage (Decode/ID) through the IF/ID pipeline register.

The IF stage components :

1) Program Counter (PC):

Holds the address of the current instruction to fetch. It is updated every cycle unless a stall occurs.

2) Instruction Memory (InstMem):

Stores the program instructions. It outputs the instruction located at the address provided by the PC.

3) Adder (PC + 1):

Adds a constant 1 to PC to form NPC, the address of the next sequential instruction.

4) PC Control (PC Ctrl):

Generates the selection signals that decide which address becomes the next PC (normal next, jump target)based on conditions such as zero/negative/positive or jump signals.

5) MUX (PcSelec):

Selects the source of the next PC. One input is the normal sequential address (PC+1/NPC), and the other inputs are alternative addresses such as jump/branch targets.

6) IF/ID Pipeline Register:

Captures and stores INST and NPC at the clock edge, then forwards them to the Decode stage.

2.2 Instruction Decode Stage (ID)

The Instruction Decode (ID) stage translates the fetched instruction into the control signals and operands required by the pipeline stages. In this stage, The instruction is split into fields such as accessed to read source operands, and immediate values are extended to 32 bits for ALU and address calculations. In addition, the ID stage supports safe pipeline operation by interacting with hazard handling logic such as stalling and forwarding, ensuring correct execution when data dependencies occur.

Major components in the ID stage

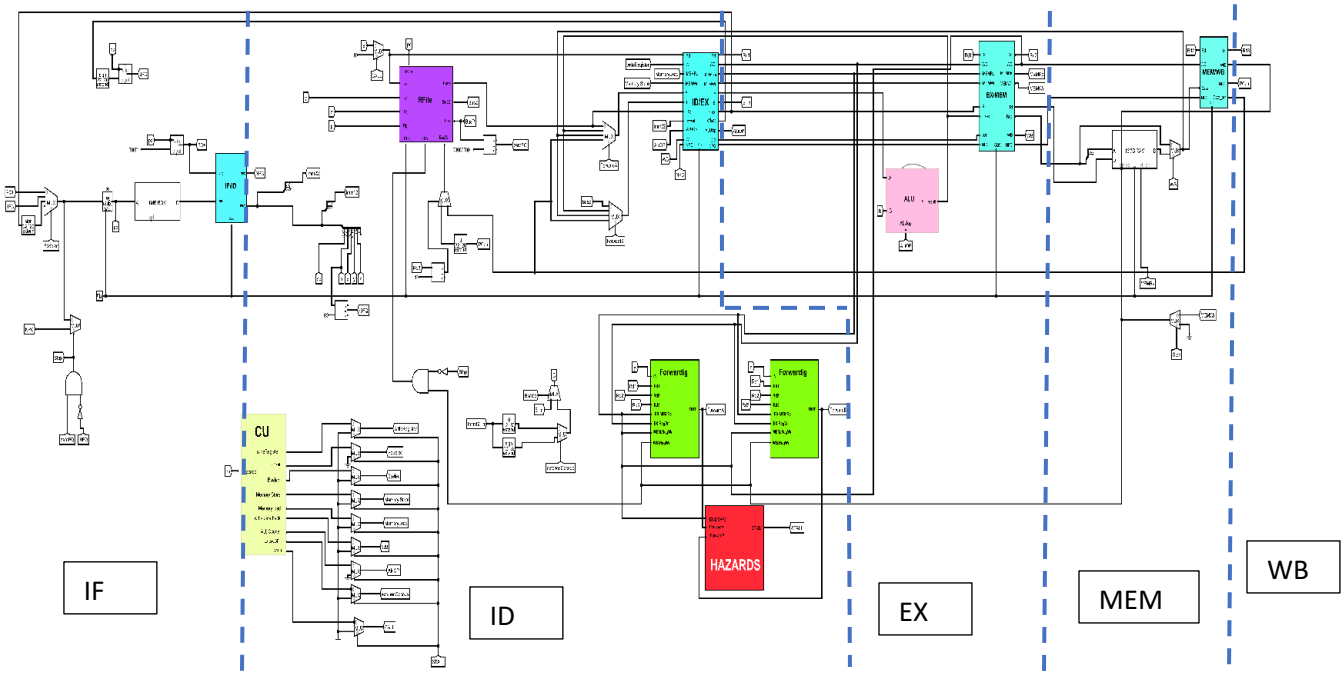


Figure 3: Datapath components in ID stage

1) Instruction Field Extraction

The instruction coming from the IF/ID register is split into fields such as the opcode, register indices (Rp, Rd, Rs, Rt), and immediate/offset values (imm12 for I-type instructions and offset/imm22 for jump/call). These fields decide both the required operands in the datapath and the control signals needed to execute the instruction.

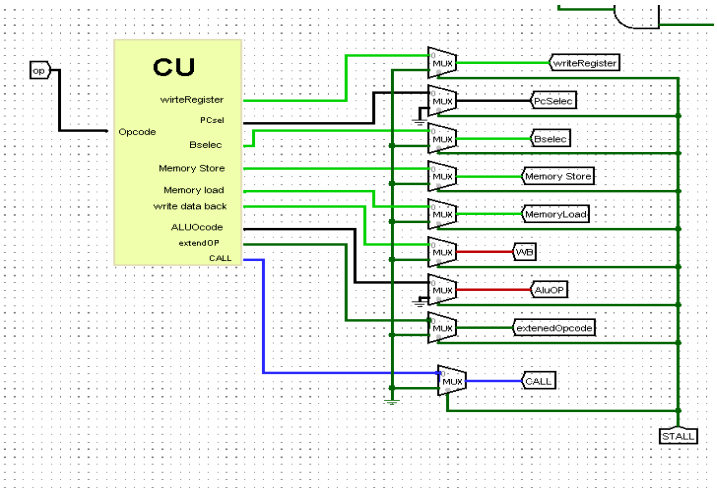


Figure 4: Instruction field extraction

2) Control Unit (CU)

The Control Unit decodes the opcode and generates the control signals that drive the datapath in later stages, such as enabling register writes, enabling memory load and store, selecting the ALU operation, choosing the ALU operand source (register or immediate), and controlling PC for jump and call behavior. The CU output is often gated during hazard conditions so that, When STALL = 1, the control unit forces the control outputs to safe values (usually 0), so the instruction behaves like a NOP: no register write, no memory read/write, and no PC redirection occur until the stall is removed.

3) Register File (RFile)

The Register File stores the processor's registers and provides the required operands during the Decode (ID) stage. It receives the register numbers extracted from the instruction through Rx, Ry, and Rz, and outputs the corresponding register values on dedicated read buses such as BusX and BusZ to be used by the following pipeline stages. Because the ISA uses predicated execution, the Register File also reads the predicate register Rp and outputs its value on BusP. This value is then compared with zero to produce the zeroEQ signal, which determines whether the instruction should execute. It also supports write-back through the BusW data input, controlled by the RW (write enable) signal, which updates the selected destination register. In addition, a mux is used on the write destination path to choose which register will be written: when CALL is asserted, the MUX selects a fixed return register (R31) so the return address can be stored correctly.

4) Predicate Evaluation (Predication using Rp)

Because the ISA uses predicated execution, the decode logic evaluates whether the instruction should execute or behave like a NOP. This is done by checking Rp (unconditional case) also the value stored in Reg[Rp] (conditional execution). If the predicate is false, the instruction's side effects are disabled (no register write, no memory write, no PC redirection).

5) Immediate / Offset Extension

This block implements the immediate/offset extension and the selection of the ALU's second operand. The 12-bit immediate field (Imm12_in) is expanded to 32 bits using two extenders: a zero extender (0→32) for logical immediates and a sign extender (sign 12→32) for arithmetic immediates and address offsets loads/stores. A mux controlled by extendedOpcode selects whether the zero extended or sign extended version should be used, depending on the instruction type. Then,

a second MUX controlled by Bselec chooses the ALU B input either from the register operand (B_in) or from the selected 32-bit extended immediate.

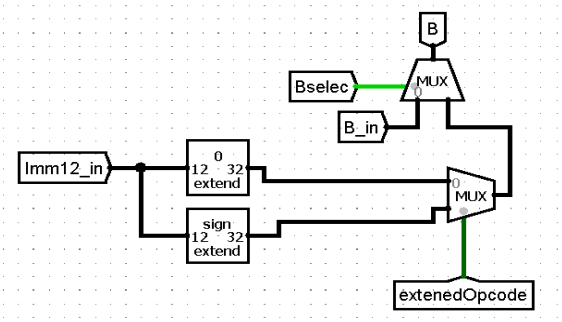


Figure 5**:** Immediate / Offset Extension

6) Forwarding and Hazards

The Forwarding and Hazards blocks work together to handle data dependencies in the pipeline so that instructions use the correct operand values without corrupting the execution. The forwarding blocks help the pipeline avoid waiting when an instruction needs a value that was produced by a previous instruction. Instead of waiting for that value to be written back into the register file, the forwarding logic checks if the current instruction's source registers match the destination register of an instruction still in the pipeline. If there is a match and the older instruction will write a result, the correct value is bypassed directly from a later stage (such as EX/MEM or MEM/WB) and the signals ForwardA and ForwardB select it for the ALU inputs.

The hazard detection block handles cases that forwarding cannot fix, especially the load use hazard. When a load (LW) is followed immediately by an instruction that uses the loaded register, the data is not ready until after the memory stage, so it cannot be forwarded in time. In this situation, the hazard unit asserts stall, which temporarily pauses the pipeline or inserts a bubble (NOP) for one cycle, ensuring the dependent instruction waits until the correct data becomes available.

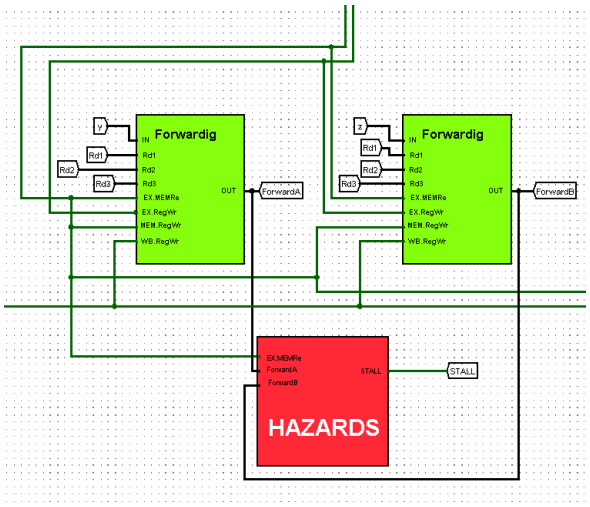


Figure 6: Forwarding and Hazards

2.2 Execution Stage

The Execution (EX) stage is responsible for performing arithmetic and logic operations based on decoded instructions. It receives control signals and operand values from the Decode stage and produces the result of the operation to be forwarded to the Memory stage. The EX stage includes key components such as the ALU, various multiplexers for operand selection, and connections for forwarding data to resolve hazards.

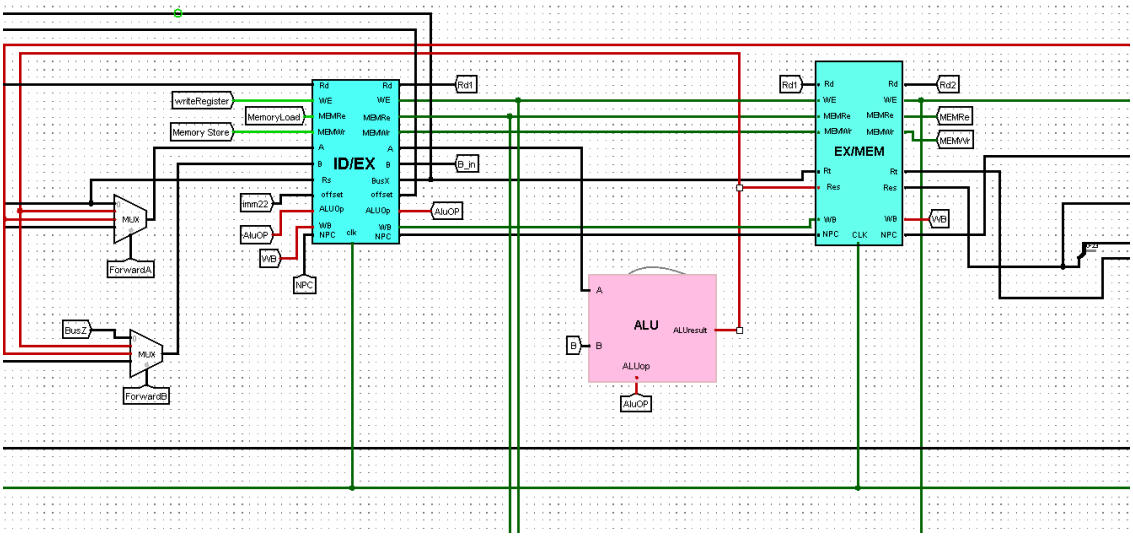


Figure 7: Execution Stage Datapath

1) ID/EX Pipeline Register:

The ID/EX pipeline register serves as the input register for the Execution stage. It holds the decoded instruction information and delivers stable values to EX at the start of the cycle, such as the source operands, the destination register fields, and the required control signals (ALUOp, WE, MEMRe, MEMWr, WB). ID/EX provides the ALU with operand lines labeled A and B in, and it forwards control signals like ALUOp and memory/write-back enables so the EX stage and the next stage operate on the correct instruction without mixing signals between cycles.

2) Operand Selection MUX (Register vs Immediate/Offset):

The operand-selection multiplexer in the EX stage ensures the ALU receives the correct second input depending on the instruction type. For R-type instructions, the ALU should use a register value (BusB), while for I-type instructions (such as ADDI, LW, and SW) the ALU must use an extended immediate/offset value. In your design, this selection happens before the ALU through the B path (B in) so the ALU can either compute a pure arithmetic/logic result or compute an effective address (Base + Offset) for memory operations.

3) ALU (Arithmetic Logic Unit):

The ALU is the core computation block of the Execution stage. It performs arithmetic and logic operations (ADD, SUB, AND, OR, NOR,) based on the ALUOp control signal, producing an output called ALUResult. For memory instructions, the ALU is also responsible for address calculation by adding the base register value to the sign-extended offset (effective address). The ALU block,receives input A and B, uses ALUOp, and outputs ALUResult which later becomes either the final result for ALU instructions or the address for LW/SW.The ALU's second input is selected according to the control signal produced in the Decode stage. Instead of re-decoding the instruction, EX simply applies this pregenerated selection to feed the ALU with the correct operand for the current instruction. This allows the ALU to either operate on two register values for R-type operations or use the forwarded/extended offset value for address-based computations (such as effective address generation in load/store), ensuring the EX stage performs the intended computation while keeping the pipeline stages clearly separated.

4) EX/MEM Pipeline Register:

The EX/MEM pipeline register stores the outputs of the Execution stage so that the Memory stage can use them in the next cycle. This includes the computed ALUResult, any data that must continue forward (such as store data for SW), the destination register identifier, and the control signals that determine whether memory or write-back actions should happen.

2.3 Memory Access

The Memory Access (MEM) stage handles all load and store operations and prepares the correct value that will be sent to the Write Back stage. The MEM stage receives the effective address from the EX/MEM pipeline register (this step was previously computed by the ALU in the EX stage)Using the control signals carried in EX/MEM mainly MEMRe (memory read enable) and MEMWr (memory write enable) the data memory either reads a word from the computed address or writes a word to that address (for load/ store instructions). If the instruction is a store, the value that gets written to memory comes from the data path forwarded through EX/MEM (the store data line).

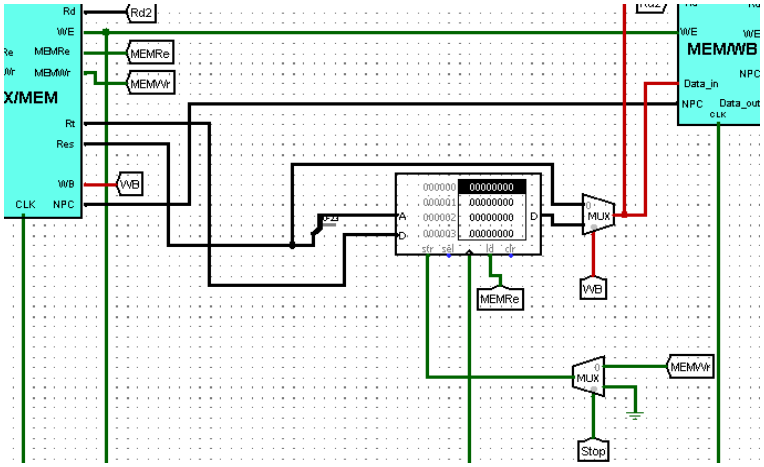


Figure 8: Memory Access Datapath

It takes: an address input from EX/MEM (the ALUResult/effective address),a data input (store data) used only when storing, and control inputs MEMRe and MEMWr to determine the operation. When MEMRe = 1, the memory outputs data_out (the loaded word). When MEMWr = 1, the memory writes Data_in into the addressed location. MEMWr is passed through a MUX controlled by Stop, so if a stall occurs, MEMWr can be forced to 0 to prevent any accidental memory write during pipeline stalls.

Write-Back selection MUX (WB MUX)

Mem stage uses a 2-to-1 mux controlled by the wb signal. This mux selects what value should continue toward the register file:Alu/address/result path (for arithmetic/logic instructions), or data_out from memory (for load instructions).

This selected value becomes the write-back data that will later be written into the destination register.

MEM/WB Pipeline Register

The mem/wb pipeline register stores the outputs produced in the mem stage so they remain stable for the next cycle. It holds the selected write-back value from the wb mux, the destination register rd, and the required control signal we. This ensures the write-back stage receives the correct data and control information and writes the result to the correct register at the proper time.

2.4 Memory WriteBack

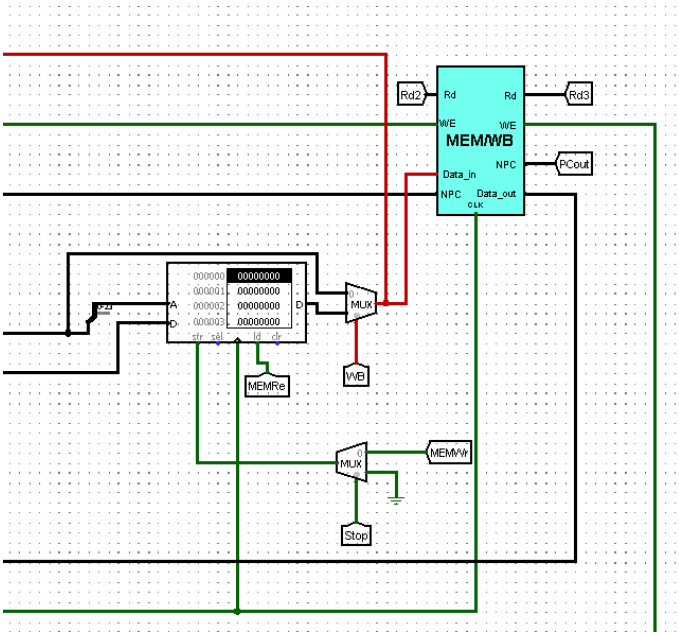


Figure 9: Memory Write Back datapath

In the Write Back (WB) stage, the processor updates the register file with the final result of the instruction. The value to be written back is already selected earlier using the WB MUX (controlled by the WB signal), which chooses between the ALU result path and the data read from memory for load instructions. The selected output is then stored inside the MEM/WB pipeline register as Data_out, along with the destination register field Rd and the write enable signal WE. During WB, when WE = 1, the register file writes Data_out into the destination register specified by Rd. Meanwhile, the NPC value is also carried through the MEM/WB register (shown as NPC/PCout) to keep program-flow information available for control/forwarding needs, but it is not written through a separate "PC write-back" path in this diagram.

3. Control Signals

3.1 Truth Table

Op	Inst.	WE	MEMRe	MEMWr	BSelect	WB	ExtOp	CALL	ALUOp	PCselec
0	ADD	1	0	0	0	0	X	0	000	00
1	SUB	1	0	0	0	0	X	0	001	00
2	OR	1	0	0	0	0	X	0	010	00
3	NOR	1	0	0	0	0	X	0	011	00
4	AND	1	0	0	0	0	X	0	100	00
5	ADDI	1	0	0	1	0	0	0	000	00
6	ORI	1	0	0	1	0	1	0	010	00
7	NORI	1	0	0	1	0	1	0	011	00
8	ANDI	1	0	0	1	0	1	0	100	00
9	LW	1	1	0	1	1	0	0	000	00
10	SW	0	0	1	1	X	0	0	000	00
11	J	0	0	0	X	X	X	0	XXX	01
12	CALL	1	0	0	X	X	X	1	XXX	01
13	JR	0	0	0	X	X	X	0	XXX	10

Table 2: Control signals truth table

3.2 Boolean Expressions

Control Signal	Boolean Expression
WE	ADD || SUB || OR || NOR || AND || ADDI || ORI || NORI || ANDI || LW || CALL
MEMRe	LW
MEMWr	SW
BSelect	ADDI || ORI || NORI || ANDI || LW || SW
WB	LW
ExOp	ADD || SUB || NOR || AND || ADDI || LW || SW || J || CALL || JR
CALL	CALL
ALUOp[0]	SUB || NOR || NORI
ALUOp[1]	OR || NOR || ORI || NORI
ALUOp[2]	AND || ANDI
PCselec[0]	J || CALL
PCselec[1]	JR

Table 3: Boolean expressions control signals

SIGANL	BOOLEAN EXPRESSION
ForwardA/B = 0	ForwardA/B != 1 && ForwardA/B!= 2 && ForwardA/B!= 3
ForwardA/B = 1	RegWrite_MEM && (Rd_MEM != 0) && (Rs_EX == Rd_MEM) && !MemToReg_MEM
ForwardA/B = 2	RegWrite_MEM && (Rd_MEM != 0) && (Rs_EX == Rd_MEM) && MemToReg_MEM
ForwardA/B = 3	!RegWrite_MEM && RegWrite_WB && (Rd_WB != 0) && (Rs_EX == Rd_WB) || RegWrite_MEM && (Rd_MEM != Rs_EX) && RegWrite_WB && (Rd_WB != 0) && (Rs_EX == Rd_WB)

Table 4: Boolean expressions Forwarding

3.3 Control Signals Description

Signal	0	1
WE	Don't write to register file	Write result to register file
MEMRe	Don't read from memory	Read from data memory
MEMWr	Don't write to memory	Write to data memory
BSelect	ALU input B = Register	ALU input B = Immediate
WB	ALU result	Memory data
ExOp	Zero-extend immediate	Sign-extend immediate
CALL	Normal instruction CALL instruction (save return address)	CALL instruction (save return address)
ALUOp[2:0]	000=ADD, 001=SUB, 010=OR, 011=NOR, 100=AND	
PCsel[1:0]	01 = PC + offset (J/CALL), 10 = Register (JR)	

Table 5: Control signals discription

3.1 State diagram

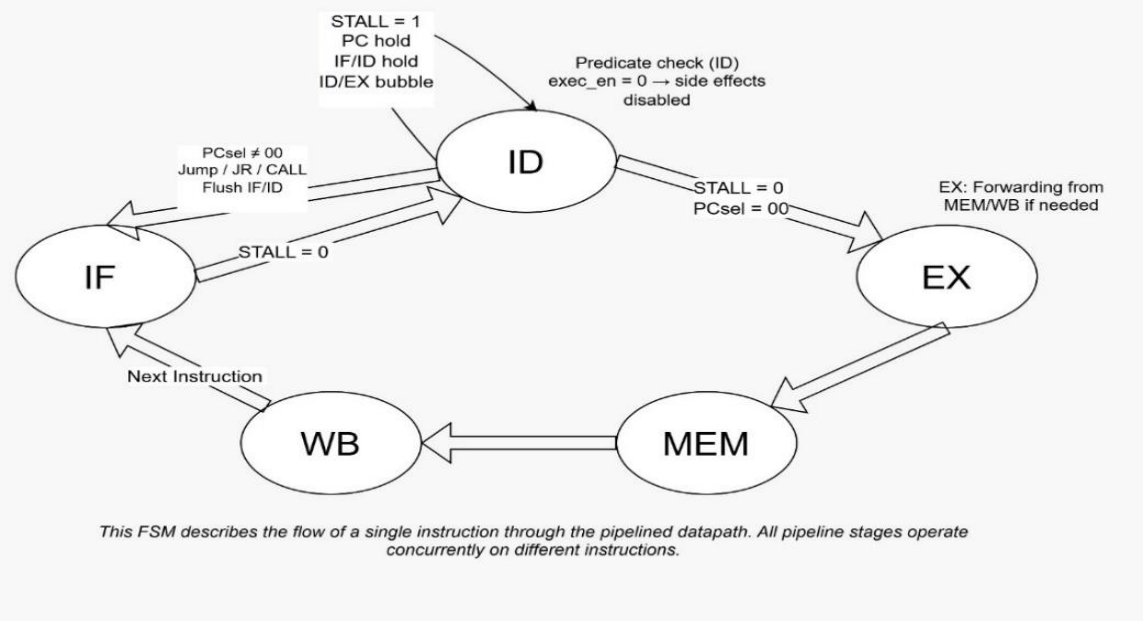


table 6: State diagram

4. Verilog Codes

4.1 Datapath Modules

ALU: The Arithmetic Logic Unit (ALU) is responsible for performing arithmetic and logical operations required by the processor during the execute stage. It takes two 32-bit operands as inputs and produces a 32-bit result based on the control signal AluOp. The ALU supports basic arithmetic operations such as addition and subtraction, as well as logical operations including AND, OR, and NOR. The specific operation is selected using a 3-bit control signal generated by the control unit. This module operates combinational, ensuring that the results are computed within a single clock cycle and forwarded to the next pipeline stage for further processing.

```
// EX Stage: ALU
ALU alu (
    .A(ALU_A),
    .B(ALU_B),
    .AluOp(ALUOpcode_EX),
    .ALU_result(ALU_result_EX)
);
```

Figure 10: ALU Code

Register File: The register file stores 32 general purpose 32-bit registers and provides two read ports and one write. Register R0 is hardwired to zero, while register R30 always reflects the current PC. Write occurs synchronously on the rising edge when write enable is active. Read operations are combinational, allowing immediate access to register values required in decode stage.

```
regfile reg_file (
    .clk(clk),
    .we(RegWrite_WB),
    .wa(FinalWriteAddr),
    .wd(FinalWriteData),
    .ra_rs(Rs_ID),
    .ra_rt(Rt_read_addr),
    .ra_rp(Rp_ID),
    .pc_value(PC_ID),
    .rd_rs(BusA_ID),
    .rd_rt(BusB_ID),
    .rd_rp(Rp_value_ID)
);
```

Figure 11: Reg file Code

Instruction Memory: This module implements a 1024-word instruction memory, where each instruction is 32 bits wide. The instruction is read combinational using the lower bits of the program counter as the address. At initialization, all memory locations are filled with NOP instructions to ensure safe execution before loading the actual program.

```
// IF Stage: Instruction Memory
instruction_memory imem (
    .address(PC),
    .instruction(instruction_IF)
);
```

Figure 12: Instruction memory Code

Data Memory: The module implements 1024-words data memory used for load and store instructions. Memory reads are performed combinational when mem_read is enabled, while writes occur synchronously on the rising edge clock edge when mem_write is active. The memory is initialized to zero, and debug messages are included to display read and write operations during simulation.

```
// MEM Stage: Data Memory
data_memory dmem (
    .clk(clk),
    .address(ALU_result_MEM),
    .write_data(StoreData_MEM),
    .mem_write(MemWrite_MEM),
    .mem_read(MemRead_MEM),
    .read_data(MemData_MEM)
);
```

Figure 13: Data Memory Code

PC_Next /pc_reg: The PC_next module computes the next program counter value based on the control signal PCsel. It supports sequential execution (PC + 1), jump instructions using a sign-extended offset, and register-based jumps (JR). The pc_reg module stores the current PC value and updates it on the rising clock edge when enabled, or resets it to zero during reset.

```
// IF Stage: PC Register
pc_reg pc_register (
    .clk(clk),
    .reset(reset),
    .next_pc(PC_next),
    .PC_en(PC_en),
    .pc(PC)
);
```

Figure 14: PC Code

4.2 Pipeline Registers

IF/ID: This module implements the pipeline register between the Instruction Fetch (IF) and Instruction Decode (ID) stages. It stores the current PC and instruction on each clock cycle. On reset, or when a control-flow instruction is taken (Kill1), a NOP is inserted to flush the pipeline. When a stall is asserted, the register holds its current values.

```
// IF/ID Pipeline Register
IF_ID if_id_reg (
    .clk(clk),
    .reset(reset),
    .stall(IF_ID_stall),
    .kill1(kill1),
    .PC_in(PC),
    .instruction_in(instruction_IF),
    .PC_out(PC_ID),
    .instruction_out(instruction_ID)
);
```

Figure 15: IF_ID Code

ID/EX: This module implements the pipeline register between the Instruction Decode (ID) and Execute (EX) stages. It transfers both control signals and data signals needed by the ALU and memory stages. On reset or when a flush is asserted, the module inserts a bubble by clearing all control signals. Otherwise, it forwards register operands, immediate values, destination registers, and control signals to the EX stage.

```
// ID/EX Pipeline Register
ID_EX id_ex_reg (
    .clk(clk),
    .reset(reset),
    .flush(flush_ID_EX),
    .ALUOp_in(ALUOpcode_ID),
    .ALUSrc_in(Bselect_ID),
    .MemRead_in(MemoryLoad_gated),
    .MemWrite_in(MemoryStore_gated),
    .WBdata_in(writeDataBack_ID),
    .RegWrite_in(writeRegister_gated),
    .call_in(call_gated),
    .BusA_in(BusA_ID),
    .BusB_in(BusB_ID),
    .imm_ext_in(imm_ext_ID),
    .Rd_in(Rd_ID),
    .PC_in(PC_ID),
    .Rs_in(Rs_ID),
    .Rt_in(Rt_read_addr),
    .ALUOp_out(ALUOpcode_EX),
    .ALUSrc_out(ALUSrc_EX),
    .MemRead_out(MemRead_EX),
    .MemWrite_out(MemWrite_EX),
    .WBdata_out(MemToReg_EX),
    .RegWrite_out(RegWrite_EX),
    .call_out(call_EX),
    .BusA_out(BusA_EX),
    .BusB_out(BusB_EX),
    .imm_ext_out(imm_ext_EX),
    .Rd_out(Rd_EX),
    .Rs_out(Rs_EX),
    .Rt_out(Rt_EX),
    .PC_out(PC_EX)
);
```

Figure 16: ID_EX Code

EX/MEM: This modul implements the pipeline register between the Execute (EX) and Memory (MEM) stages. It forwards the ALU result, store data, destination register, and all memory and write-back control signals. It also carries the next PC value and CALL control signal. On reset, all outputs are cleared to insert a safe pipeline state.

```
// EX/MEM Pipeline Register
EX_MEM ex_mem_reg (
    .clk(clk),
    .reset(reset),
    .MemRead_in(MemRead_EX),
    .MemWrite_in(MemWrite_EX),
    .MemToReg_in(MemToReg_EX),
    .RegWrite_in(RegWrite_EX),
    .call_in(call_EX),
    .NPC_in(NPC_EX),
    .ALU_result_in(ALU_result_EX),
    .StoreData_in(StoreData_EX),
    .Rd_in(Rd_EX),
    .MemRead_out(MemRead_MEM),
    .MemWrite_out(MemWrite_MEM),
    .MemToReg_out(MemToReg_MEM),
    .RegWrite_out(RegWrite_MEM),
    .call_out(call_MEM),
    .NPC_out(NPC_MEM),
    .ALU_result_out(ALU_result_MEM),
    .StoreData_out(StoreData_MEM),
    .Rd_out(Rd_MEM)
);
```

Figure 17: EX_MEM Code

MEM/WB: The module implements the pipeline registers between the Memory (MEM) and Write Back (WB) stages. It forwards the destination registers, write_enable signal, data to be written back, and CALL related signals. On reset, all outputs are cleared to ensure correct pipeline initialization before execution.

```
// MEM/WB Pipeline Register
MEM_WB mem_wb_reg (
    .clk(clk),
    .reset(reset),
    .Rd_in(Rd_MEM),
    .WE_in(RegWrite_MEM),
    .DATA_in(MEM_WB_Data),
    .NPC_in(NPC_MEM),
    .call_in(call_MEM),
    .Rd_out(Rd_WB),
    .WE_out(RegWrite_WB),
    .DATA_out(WriteData_WB),
    .call_out(call_WB),
    .NPC_out(NPC_WB)
);
```

Figure 18: MEM_WB Code

4.3 Control and Hazard Handling

Control Unit: This module generates the processor control signals based on the 5-bit OpCode. It selects the ALU operations, choose between register/immediate input (Bselect), controls memory load/store signals, and enables register write-back. It also handles control-flow instruction by setting PCsel for Jump and JR, and asserts call for CALL instructions. The extendOP signal decide whether the immediate is sign-extended or zero-extended.

```
// ID Stage: Control Unit
control_unit ctrl_unit (
    .Opcode(opcode_ID),
    .writeRegister(writeRegister_ID),
    .PCsel(PCsel_ID),
    .Bselect(Bselect_ID),
    .MemoryStore(MemoryStore_ID),
    .MemoryLoad(MemoryLoad_ID),
    .writeDataBack(writeDataBack_ID),
    .ALUOpcode(ALUOpcode_ID),
    .extendOP(extendOP_ID),
    .call(call_ID)
);
```

Figure 19: Control Unit Code

Forwarding Unit: This module resolves data hazards by forwarding results from later pipeline stages to the Execute stage. If the required operand is available in the MEM or WB stage, the forwarding logic selects the correct source instead of waiting for write-back.

```
// EX Stage: Forwarding Unit A
    Forwarding forward_unit_A (
    .Rs_EX(Rs_EX),
    .Rd_MEM(MEM_WriteAddr),
    .Rd_WB(FinalWriteAddr),
    .RegWrite_MEM(RegWrite_MEM),
    .MemToReg_MEM(MemToReg_MEM),
    .RegWrite_WB(RegWrite_WB),
    .Forward(ForwardA)
    );

    Forwarding forward_unit_B (
    .Rs_EX(Rt_EX),
    .Rd_MEM(MEM_WriteAddr),
    .Rd_WB(FinalWriteAddr),
    .RegWrite_MEM(RegWrite_MEM),
    .MemToReg_MEM(MemToReg_MEM),
    .RegWrite_WB(RegWrite_WB),
    .Forward(ForwardB)
    );
```

Figure 20: Forwarding Code

Hazards Detection Unit: This module detects load-use hazards. When an instruction in the EX stage is reading from memory and its destination register is needed by the following instruction, a stall signal is generated to pause the pipeline and prevent incorrect execution.

```
// ID Stage: Hazard Detection Unit
Hazards hazard_unit (
    .MemRead_EX(MemRead_EX),
    .Rd_EX(Rd_EX),
    .MemToReg_WB(MemToReg_WB),
    .Rd_WB(FinalWriteAddr),
    .Rs_ID(Rs_ID),
    .Rt_ID(Rt_read_addr),
    .Rp_ID(Rp_ID),
    .PCsel_ID(PCsel_ID),
    .RegWrite_EX(RegWrite_EX),
    .STALL(STALL)
);
```

Figure 21: Hazards Code

Predicate Unit: This module controls predicated execution. An instruction is allowed to execute if the predicate register is R0 or the value of the predicate register is non-zero. Otherwise, the instruction's side effects are disabled.

```
// ID Stage: Predicate Unit
predicate_unit pred_unit (
    .rp_addr(Rp_ID),
    .rp_val(Rp_value_forwarded),
    .exec_en(exec_en_ID)
);
```

Figure 22: Predicate Unit Code

Top Module: The processor_top module connects all processor components to form a 5-stage pipelined processor (IF, ID, EX, MEM, WB) **IF stage** fetches the instruction using the program counter and instruction memory.

- ID stage decodes the instruction, generates control signals, reads registers, and checks the predicate condition.
- EX stage executes the instruction using the ALU and applies forwarding when needed.
- MEM stage performs memory read or write for load and store instructions.
- WB stage writes the final result back to the register file, including handling the CALL instruction.

Stall, forwarding, flushing, and predication are used to ensure correct execution in the presence of hazards and control-flow changes.

5. Test and Verification

Several Instructions and cases were operated on ACTIVE HDL to test the code's functionality. According to the ISA provided in the project, the instructions were classified into 4 groups written in four different test benches: Arithmetic and Logical, Memory, Control flow instructions and hazard handling cases. The results were shown through waveform diagrams and debugging displayed statements.

Below are different instructions with their Binary code and Function that covers the ISA.

No.	Instructions	Binary Code	Function
[0]	ADDI R1, R0, 5	00101_00000_00001_00000_000000000101	Initialize R1 with the value 5.
[1]	ADDI R2, R0, 3	00101_00000_00010_00000_000000000011	Initialize R2 with the value 3.
[2]	ADD R3, R1, R2	00000_00000_00011_00001_00010_0000000	R3 = 5 + 3 = 8.
[3]	SUB R4, R3, R2	00001_00000_00100_00011_00010_0000000	R4 = 8 - 3 = 5.
[4]	ADDI R5, R1, 10	00101_00000_00101_00001_000000001010	R5 = 5 + 10 = 15.
[5]	OR R6, R1, R2	00010_00000_00110_00001_00010_0000000	R6 = 5 | 3 = 7.
[6]	AND R7, R1, R2	00100_00000_00111_00001_00010_0000000	R7 = 5 & 3 = 1.
[10]	ORI R10, R1, 0F	00110_00000_01010_00001_000000001111	R10 = 5 | 15 = 15.
[11]	ANDI R11, R1, 0F	01001_00000_01011_00001_000000001111	R11 = 5 & 15 = 5.
[12]	NORI R12, R1, 0F	00111_00000_01100_00001_000000001111	R12 = ~(5 | 15).
[13]	ADDI R3, R0, 100	00101_00000_00011_00000_000001100100	Initialize R3 with 100.
[14]	SW R3, 10(R0)	01011_00000_00011_00000_000000001010	Store value of R3 (100) into Mem[10].
[15]	LW R8, 10(R0)	01010_00000_01000_00000_000000001010	Load value from Mem[10] (100) into R8.
[16]	ADDI R9, R8, 2	00101_00000_01001_01000_000000000010	R9 = 100 + 2 = 102.
[17]	ADDI R11, R0, 2	00101_01001_01011_00000_000000000010	Initialize R11 with 2.
[18]	J 3	01100_00000_00000000000000000000011	Jump to Address 3.
[19]	ADDI R12, R0, 3	00101_00000_01100_00000_000000000011	(Flushed if J 3 is taken).
[22]	CALL 3	01101_00000_00000000000000000000011	Call subroutine at Address 3.
[24]	ADDI R7, R0, 2	00101_00000_00111_00000_000000000010	Initialize R7 with 2.
[25]	JR R7	01110_00000_00000_00111_00000_0000000	Jump to instruction at Address in R7 (Address 2).

Table 6: Instructions

5.1 Test 1: Arithmetic and logic instructions

- **Instruction**

```
// ---- Arithmetic & Logic ----
$display("ALU Ops...");
uut.imem.memory[0]  = {5'd5, 5'd0, 5'd1, 5'd0, 12'd5};     // ADDI R1, R0, 5
uut.imem.memory[1]  = {5'd5, 5'd0, 5'd2, 5'd0, 12'd3};     // ADDI R2, R0, 3
uut.imem.memory[2]  = {5'd0, 5'd0, 5'd3, 5'd1, 5'd2, 7'd0}; // ADD R3,R1,R2
uut.imem.memory[3]  = {5'd1, 5'd0, 5'd4, 5'd3, 5'd2, 7'd0}; // SUB R4,R3,R2
uut.imem.memory[4]  = {5'd5, 5'd0, 5'd5, 5'd1, 12'd10};    // ADDI R5,R1,10
uut.imem.memory[5]  = {5'd2, 5'd0, 5'd6, 5'd1, 5'd2, 7'd0}; // OR  R6,R1,R2
uut.imem.memory[6]  = {5'd4, 5'd0, 5'd7, 5'd1, 5'd2, 7'd0}; // AND R7,R1,R2
uut.imem.memory[10] = {5'd6, 5'd0, 5'd10,5'd1, 12'h00F};   // ORI  R10,R1,0F
uut.imem.memory[11] = {5'd9, 5'd0, 5'd11,5'd1, 12'h00F};   // ANDI R11,R1,0F
uut.imem.memory[12] = {5'd7, 5'd0, 5'd12,5'd1, 12'h00F};   // NORI R12,R1,OF
```

Figure 23: Test 1 Code

- **Results**

```
# KERNEL: ALU Ops...
# KERNEL:
# KERNEL: CYCLE    | PC    | IF      | ID      | STALL | EVENTS
# KERNEL: ------------------------------------------------------------
# KERNEL: 1        | 1     | ADDI    | ADDI    | 0     |
# KERNEL: 2        | 2     | ADD     | ADDI    | 0     |
# KERNEL: 3        | 3     | SUB     | ADD     | 0     | [REG_WR]
# KERNEL: 4        | 4     | ADDI    | SUB     | 0     | [REG_WR]
# KERNEL: 5        | 5     | OR      | ADDI    | 0     | [REG_WR]
# KERNEL: 6        | 6     | AND     | OR      | 0     | [REG_WR]
# KERNEL: 7        | 7     | NOP     | AND     | 0     | [REG_WR]
# KERNEL: 8        | 8     | NOP     | NOP     | 0     | [REG_WR]
# KERNEL: 9        | 9     | NOP     | NOP     | 0     | [REG_WR]
# KERNEL: 10       | 10    | ORI     | NOP     | 0     | [REG_WR]
# KERNEL: 11       | 11    | ANDI    | ORI     | 0     | [REG_WR]
# KERNEL: 12       | 12    | NORI    | ANDI    | 0     | [REG_WR]
# KERNEL: 13       | 13    | NOP     | NORI    | 0     | [REG_WR]
# KERNEL: 14       | 14    | NOP     | NOP     | 0     | [REG_WR]
# KERNEL: 15       | 15    | NOP     | NOP     | 0     | [REG_WR]
# KERNEL:
# KERNEL: ================================================
# KERNEL:                 FINAL REGISTER DUMP
# KERNEL: ================================================
# KERNEL:   R1   : 5              |  R2  : 3
# KERNEL:   R3   : 8              |  R4  : 5
# KERNEL:   R5   : 15             |  R6  : 7
# KERNEL:   R7   : 1              |  R10 : 15
# KERNEL:   R11  : 5              |  R12 : 4294967280
```

Figure 24: Test 1 result

- **WaveForm**

Figure 25: Test 1 waveform

Discussion

In this Test, we validate the functionality of the arithmetic and logical instruction set in the RISC processor. The instructions include ADD, SUB, OR, AND, NOR operations between 2 register operands or an immediate with a 5-bit predicate.

The test starts by initializing some registers by using ADDI instruction and R0 register which has a constant 0 value, then it performs several logical and arithmetic instructions. The final Results matched the expected ones.

The cycle-by-cycle results show that control signals propagate through the pipeline stages exactly as expected. As shown in the results, instructions progress smoothly through the pipeline without stalls. The waveform shows stable PC updates, correct instruction fetches and decode, correct control signal generation.

5.2 Test 2: Memory Access

- **Instructions**

```
$display("Memory Ops...");
uut.imem.memory[0]  = {5'd5,5'd0, 5'd3, 5'd0, 12'd100};     // ADD R3, R0, 100
uut.imem.memory[1]  = {5'd11,5'd0, 5'd3, 5'd0, 12'd10};    // SW R3 into Mem[10]
uut.imem.memory[2]  = {5'd10, 5'd0, 5'd8, 5'd0, 12'd10};   // LW Mem[10] into R8
uut.imem.memory[3]  = {5'd5, 5'd0, 5'd9, 5'd8, 12'd2};     // ADDI R9, R8, 2
uut.imem.memory[4]  = {5'd5, 5'd9, 5'd11, 5'd0, 12'd2};       //ADDI R11, R0, 2
```

Figure 26: Test 2 Code

- **Results**

```
# KERNEL: Memory Ops...
# KERNEL:
# KERNEL: CYCLE    | PC    | IF      | ID      | STALL | EVENTS
# KERNEL: ------------------------------------------------------------
# KERNEL: 1        | 1     | SW      | ADDI    | 0     |
# KERNEL: 2        | 2     | LW      | SW      | 0     |
# KERNEL: 3        | 3     | ADDI    | LW      | 0     | [REG_WR]
# KERNEL: 4        | 4     | ADDI    | ADDI    | 1     | [HAZARD!] [REG_WR] [MEM_WR]
# KERNEL: >>>>>> MEMWRITE : WordAddr=10 (0x0000000a)(Data=0x00000064)<<<<<
# KERNEL: 5        | 4     | ADDI    | ADDI    | 0     |
# KERNEL: >>>>>> MEMREAD : WordAddr=10 (0x0000000a)(Data=0x00000064)<<<<<<
# KERNEL: 6        | 5     | NOP     | ADDI    | 0     | [REG_WR]
# KERNEL: 7        | 6     | NOP     | NOP     | 0     |
# KERNEL: 8        | 7     | NOP     | NOP     | 0     | [REG_WR]
# KERNEL: 9        | 8     | NOP     | NOP     | 0     |
# KERNEL:
# KERNEL: ================================================
# KERNEL:                 FINAL REGISTER
# KERNEL: ================================================
# KERNEL:   R3   : 100
# KERNEL:   R8   : 100
# KERNEL:   R9   : 102
# KERNEL:  R11   : 0
# KERNEL:
# KERNEL: ================================================
# KERNEL:                 FINAL MEMORY DUMP
# KERNEL: ================================================
# KERNEL:   Mem[10] : 100
```

Figure 27: Test 2 results

- **WaveForm**

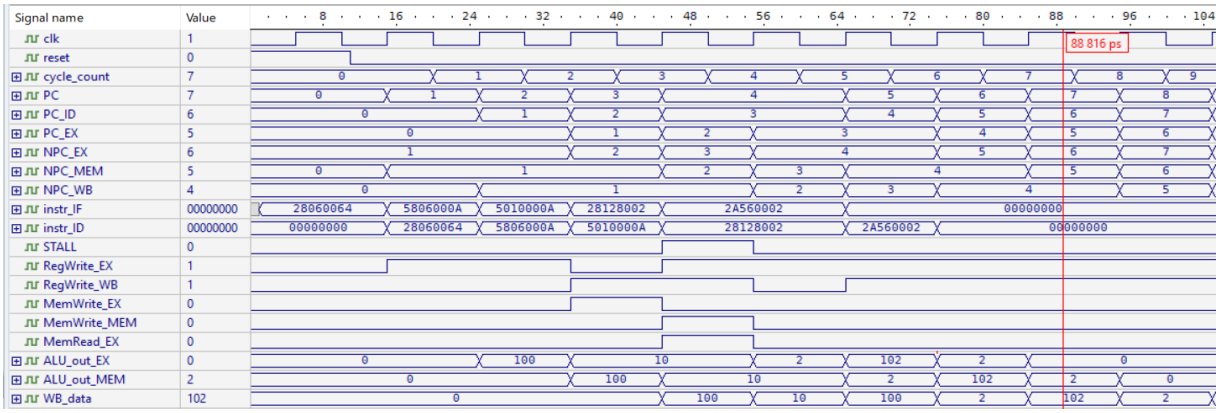


Figure 28: Test 2 Waveform

Discussion

This test verifies how the processor handles memory access instructions (LW and SW) and their interaction with the pipelined datapath. The program starts by initializing R3, then to test it stores the value of R3 into memory and then loading it back into R9. By these instructions, we effectively tested the hardware's ability to calculate addresses, manage memory read/write controls, and handle the write-back process in a pipelined.

The cycle-by-cycle results show that control signals propagate through the pipeline stages exactly as expected. During the store operation, the MemWrite signal is correctly asserted in the MEM stage. Whereas for the load instruction, MemRead and RegWrite activate in their respective stages, allowing the data to be retrieved and forwarded.

Also, another case was tested in instruction 4, where the predicate test fails and the instruction doesn't operate. Rp's value is 9, which has zero value, so after checking the register, this cycle fails and doesn't execute due to false predicate.

The waveforms validate that the pipeline is functioning smoothly without any unnecessary stalls. We can see the instructions overlapping correctly across the IF, ID, EX, MEM, and WB stages while the Program Counter increments stably. The final Results displayed through running matched the expected ones, which indicates the success of the operations.

5.3 Test 3: Control Flow Instructions

- **Instructions**

```
// ------ Control Flow Instructions ------
uut.imem.memory[0] = {5'd12, 5'd0, 22'd3};                    // J 3
uut.imem.memory[1] = {5'd5, 5'd0, 5'd12, 5'd0, 12'd3};        // ADDI R12,R0,3
uut.imem.memory[2] = {5'd5, 5'd0, 5'd13, 5'd0, 12'd111};      // ADDI R13,R0,111
uut.imem.memory[3] = {5'd5, 5'd0, 5'd13, 5'd0, 12'd222};      // ADDI R13,R0,222
uut.imem.memory[4] = {5'd13, 5'd0, 22'd3};                    // CALL 3
uut.imem.memory[5] = {5'd5, 5'd0, 5'd14, 5'd0, 12'd333};      // ADDI R14,R0,333
uut.imem.memory[7] = {5'd5, 5'd0, 5'd7, 5'd0, 12'd2};         // ADDI R7,R0,2
uut.imem.memory[9] = {5'd14, 5'd0, 5'd0, 5'd7, 5'd0, 7'd0};   // JR R7
```

Figure 29: Test 3 Code

- **Results**

```
# KERNEL:        CONTROL FLOW INSTRUCTIONS
# KERNEL: CYCLE    | PC    | IF      | ID      | STALL | EVENTS
# KERNEL: ---------------------------------------------------------------
# KERNEL: 1        | 1     | ADDI    | J       | 0     | [FLUSH]
# KERNEL: 2        | 3     | ADDI    | NOP     | 0     |
# KERNEL: 3        | 4     | CALL    | ADDI    | 0     |
# KERNEL: 4        | 5     | ADDI    | CALL    | 0     | [FLUSH]
# KERNEL: 5        | 7     | ADDI    | NOP     | 0     |
# KERNEL: 6        | 8     | NOP     | ADDI    | 0     | [WB: R13=222]
# KERNEL: 7        | 9     | JR      | NOP     | 0     | [WB: R31=5]
# KERNEL: 8        | 10    | NOP     | JR      | 0     | [FLUSH]
# KERNEL: 9        | 1     | ADDI    | NOP     | 0     | [WB: R7=1]
# KERNEL: 10       | 2     | ADDI    | ADDI    | 0     |
# KERNEL: 11       | 3     | ADDI    | ADDI    | 0     |
# KERNEL: 12       | 4     | CALL    | ADDI    | 0     |
# KERNEL: 13       | 5     | ADDI    | CALL    | 0     | [FLUSH] [WB: R12=3]
# KERNEL: 14       | 7     | ADDI    | NOP     | 0     | [WB: R13=111]
# KERNEL: 15       | 8     | NOP     | ADDI    | 0     | [WB: R13=222]
# KERNEL: 16       | 9     | JR      | NOP     | 0     | [WB: R31=5]
# KERNEL: 17       | 10    | NOP     | JR      | 0     | [FLUSH]
# KERNEL: 18       | 1     | ADDI    | NOP     | 0     | [WB: R7=1]
# KERNEL: 19       | 2     | ADDI    | ADDI    | 0     |
# KERNEL: 20       | 3     | ADDI    | ADDI    | 0     |
# KERNEL: 21       | 4     | CALL    | ADDI    | 0     |
# KERNEL: 22       | 5     | ADDI    | CALL    | 0     | [FLUSH] [WB: R12=3]
# KERNEL: 23       | 7     | ADDI    | NOP     | 0     | [WB: R13=111]
# KERNEL: 24       | 8     | NOP     | ADDI    | 0     | [WB: R13=222]
# KERNEL: 25       | 9     | JR      | NOP     | 0     | [WB: R31=5]
# KERNEL: 26       | 10    | NOP     | JR      | 0     | [FLUSH]
# KERNEL: 27       | 1     | ADDI    | NOP     | 0     | [WB: R7=1]
# KERNEL: 28       | 2     | ADDI    | ADDI    | 0     |
# KERNEL: 29       | 3     | ADDI    | ADDI    | 0     |
# KERNEL: 30       | 4     | CALL    | ADDI    | 0     |
# KERNEL:
# KERNEL: ============================================================
# KERNEL:                  FINAL REGISTER
# KERNEL: ============================================================
# KERNEL:   R7  : 1
# KERNEL:   R12 : 3
# KERNEL:   R13 : 222
# KERNEL:   R14 : 0
# KERNEL:   R30 : 4
# KERNEL:   R31 : 5
```

Figure 30: Test 3 results

- **WaveForm**

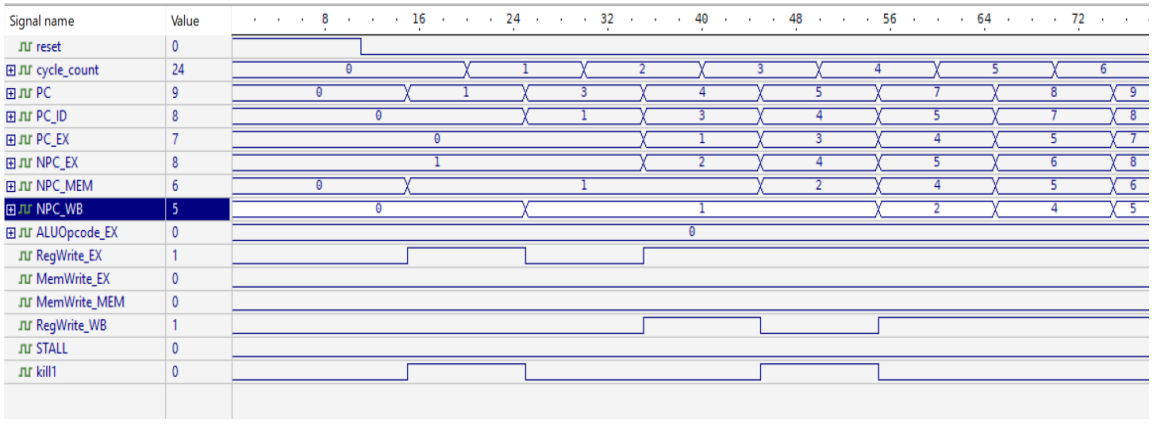


Figure 31: Test 3 Waveform

Discussion

This test checks how the processor handles control flow instructions, specifically J, CALL, and JR. It starts with using ADDI instructions that act as initializers to set up register values. The purpose is to verify that the PC updates correctly for unconditional jumps and function calls.

The cycle-by-cycle trace shows the processor correctly managing the control hazards that occur during a jump. When a J or CALL instruction reaches the execute stage, the PC control logic selects the new target address, causing the next sequentially fetched instructions to be invalid. As a result, the pipeline correctly flushes the instructions in the IF and ID stages, as indicated by the repeated FLUSH events in the results table. Additionally, the CALL instruction properly writes the return address to R31 during the write-back stage. Similarly, the JR instruction updates the PC using the value stored in a register. To confirm that we can see in the waveform how the PC jumped to the value 3 after J instructions, which avoids the instruction of address 2.

For CALL 3 instruction, it saves the address of the next PC on R31 as required. Knowing that the WriteBack operation is at stage 5, the results show how it correctly pipelined the value of NPC and wrote it on R31. Also, as it shows, R14's value remained 0 due to the function CALL that ignored the instruction 5 (by causing a kill) and continued to the target address (at address 7). At last, for JR, the PC updated to the value stored in R7 (which is 1) and went back to update on R12, as it shows in the displayed statements.

The waveform confirms correct pipelined control-flow handling, showing how the PC select and related control signals transition at the appropriate pipeline stage, ensuring that the correct target address is used. The final Results displayed through running matched the expected ones, which indicates the success of the operations.

5.4 Test 4: Possible Hazards Testing

A) Part 1:

- **Instructions**

```
uut.dmem.DataMem[10] = 32'd999;
uut.dmem.DataMem[20] = 32'd777;
uut.dmem.DataMem[30] = 32'd555;
$display("TEST 1: EX-to-EX Forwarding");
$display("Expected: R2=5, R3=8, R4=13\n");
uut.imem.memory[0] = {5'd5, 5'd0, 5'd2, 5'd0, 12'd5};           // ADDI R2, R0, 5
uut.imem.memory[1] = {5'd5, 5'd0, 5'd3, 5'd2, 12'd3};           // ADDI R3, R2, 3
uut.imem.memory[2] = {5'd0, 5'd0, 5'd4, 5'd3, 5'd2, 7'd0};      // ADD  R4, R3, R2

$display("TEST 2: Load-Use Hazard (Stall Expected)");
$display("Expected: R5=999, R6=1004\n");
uut.imem.memory[3] = {5'd10, 5'd0, 5'd5, 5'd0, 12'd10};         // LW R5, 10(R0)
uut.imem.memory[4] = {5'd5, 5'd0, 5'd6, 5'd5, 12'd5};           // ADDI R6, R5, 5

$display("TEST 3: Store Data Forwarding");
$display("Expected: Mem[50]=13\n");
uut.imem.memory[12] = {5'd11, 5'd0, 5'd4, 5'd0, 12'd50};        // SW R4, 50(R0)

$display("TEST 4: Load-to-Store Dependency");
$display("Expected: Mem[60]=777\n");
uut.imem.memory[13] = {5'd10, 5'd0, 5'd14, 5'd0, 12'd20};       // LW R14, 20(R0)
uut.imem.memory[14] = {5'd11, 5'd0, 5'd14, 5'd0, 12'd60};       // SW R14, 60(R0)

$display("TEST 5: Multiple Dependencies Chain");
$display("Expected: R15=10, R16=20, R17=30, R18=60\n");
uut.imem.memory[15] = {5'd5, 5'd0, 5'd15, 5'd0, 12'd10};        // ADDI R15, R0, 10
uut.imem.memory[16] = {5'd0, 5'd0, 5'd16, 5'd15, 5'd15, 7'd0}; // ADD R16, R15, R15
uut.imem.memory[17] = {5'd0, 5'd0, 5'd17, 5'd16, 5'd15, 7'd0}; // ADD R17, R16, R15
uut.imem.memory[18] = {5'd0, 5'd0, 5'd18, 5'd17, 5'd17, 7'd0}; // ADD R18, R17, R17

$display("TEST 6: Predicated Instruction - False Predicate");
$display("Expected: R19=0 (instruction not executed)\n");
uut.imem.memory[19] = {5'd5, 5'd1, 5'd19, 5'd0, 12'd999};       // ADDI R19, R0, 999, Rp=R1

$display("TEST 7: Predicated Instruction - True Predicate");
$display("Expected: R20=888\n");
uut.imem.memory[20] = {5'd5, 5'd2, 5'd20, 5'd0, 12'd888};       // ADDI R20, R0, 888, Rp=R2

$display("TEST 8: Predicate Register Dependency");
$display("Expected: R1=1, R21=100\n");
uut.imem.memory[21] = {5'd5, 5'd0, 5'd1, 5'd0, 12'd1};          // ADDI R1, R0, 1
uut.imem.memory[22] = 32'h0;                                     // NOP
uut.imem.memory[23] = {5'd5, 5'd1, 5'd21, 5'd0, 12'd100};       // ADDI R21, R0, 100, Rp=R1
```

Figure 32: Test 4 Code part 1

- **Results**

```
# KERNEL: TEST 1: EX-to-EX Forwarding
# KERNEL:   R2  = 5    (Expected: 5)     PASS
# KERNEL:   R3  = 8    (Expected: 8)     PASS
# KERNEL:   R4  = 13   (Expected: 13)    PASS
# KERNEL:
# KERNEL: TEST 2: Load-Use Hazard
# KERNEL:   R5  = 999  (Expected: 999)   PASS
# KERNEL:   R6  = 1004 (Expected: 1004)  PASS
# KERNEL:
# KERNEL: TEST 3: Store Data Forwarding
# KERNEL:   Mem[50] = 13   (Expected: 13)  PASS
# KERNEL:
# KERNEL: TEST 4: Load-to-Store Dependency
# KERNEL:   Mem[60] = 777  (Expected: 777)  PASS
# KERNEL:
# KERNEL: TEST 5: Multiple Dependencies Chain
# KERNEL:   R15 = 10   (Expected: 10)    PASS
# KERNEL:   R16 = 20   (Expected: 20)    PASS
# KERNEL:   R17 = 30   (Expected: 30)    PASS
# KERNEL:   R18 = 60   (Expected: 60)    PASS
# KERNEL:
# KERNEL: TEST 6: Predicated - False (R1=0)
# KERNEL:   R19 = 0    (Expected: 0)     PASS
# KERNEL:
# KERNEL: TEST 7: Predicated - True (R2=5)
# KERNEL:   R20 = 888  (Expected: 888)   PASS
# KERNEL:
# KERNEL: TEST 8: Predicate Register Dependency
# KERNEL:   R1  = 1    (Expected: 1)     PASS
# KERNEL:   R21 = 100  (Expected: 100)   PASS
```

Figure 33: Test 4 Results

- **WaveForm**

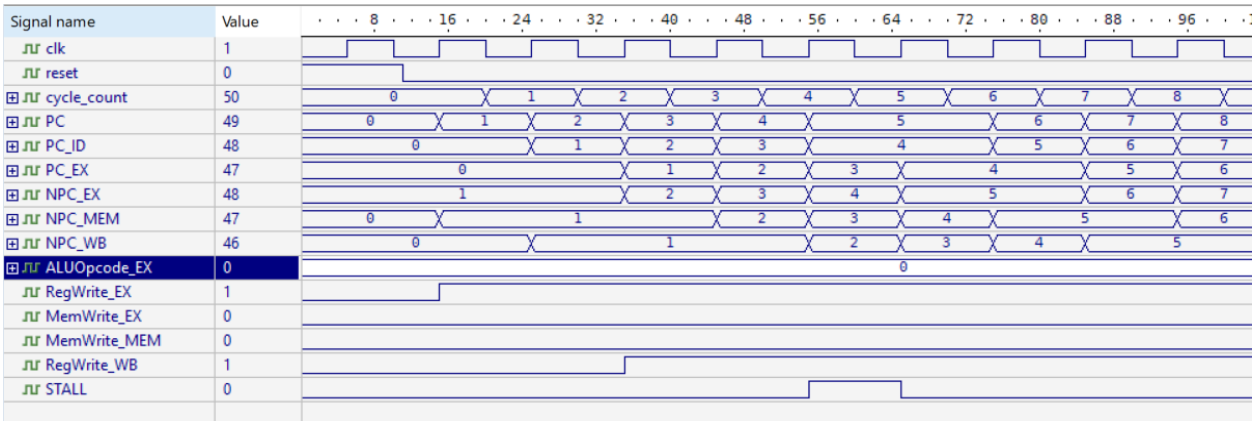


Figure 34: Test 4 waveform

Discussion

In this test, several cases were tested where possible hazards could happen in the pipelined RISC processor. This test puts the hardware through its paces by simulating high-pressure scenarios like EX-to-EX forwarding, load-use hazards, and complex dependency chains.

Each test targets a specific failure point, such as a processor accidentally using a stale register value or failing to nullify an instruction when a predicate is false.

Several of these tests focus on memory-related hazards. For example, the load-use hazard checks a case where an instruction immediately consumes the result of a load, which would normally still be in the MEM stage. Without hazard detection and stalling, the dependent instruction would read an old value from the register file. The passing result confirms that the pipeline correctly inserts a stall or bubble to wait for the load data. Similarly, the store data forwarding and load-to-store dependency tests verify that a store instruction following a producing instruction receives the correct data, even when that data has not yet been written back.

The later tests validate predicated execution, ensuring that instructions are conditionally executed based on the value of the predicate register. The predicate register dependency test confirms that predicate values themselves are correctly forwarded or resolved before being used, preventing incorrect execution decisions.

B) Part 2:

- **Instructions**

```
// Initialize data memory
uut.dmem.DataMem[10] = 32'd999;
uut.dmem.DataMem[20] = 32'd777;
uut.dmem.DataMem[30] = 32'd555;
uut.dmem.DataMem[100] = 32'd1234;
$display("TEST 9: Load with Address Dependency");
$display("Expected: R23=30, R24=555\n");
uut.imem.memory[0] = {5'd5, 5'd0, 5'd23, 5'd0, 12'd30};        // ADDI R23, R0, 30
uut.imem.memory[1] = {5'd10, 5'd0, 5'd24, 5'd23, 12'd0};       // LW R24, 0(R23)

$display("TEST 10: Store with Address and Data Dependencies");
$display("Expected: R22=42, R25=40, Mem[40]=42\n");
uut.imem.memory[2] = {5'd5, 5'd0, 5'd22, 5'd0, 12'd42};        // ADDI R22, R0, 42
uut.imem.memory[3] = {5'd5, 5'd0, 5'd25, 5'd0, 12'd40};        // ADDI R25, R0, 40
uut.imem.memory[4] = {5'd11, 5'd0, 5'd22, 5'd25, 12'd0};       // SW R22, 0(R25)

$display("TEST 11: CALL Instruction and R31 Forwarding");
$display("Expected: R31=9, R29=9\n");
uut.imem.memory[8] = {5'd13, 5'd0, 22'd3};                     // CALL 3
uut.imem.memory[9] = {5'd5, 5'd0, 5'd29, 5'd0, 12'd111};       // ADDI R29, R0, 111
uut.imem.memory[11] = {5'd5, 5'd0, 5'd29, 5'd31, 12'd0};       // ADDI R29, R31, 0

$display("\nTEST 12: R31 Dependencies");
  $display("  R19 = %-4d (Expected: 18)   %s", uut.reg_file.register[19],
  (uut.reg_file.register[19] == 18) ? "PASS ?" : "FAIL ?");
uut.imem.memory[12] = {5'd0, 5'd0, 5'd19, 5'd29, 5'd29, 7'd0};  // ADD R19, R29, R29

$display("TEST 13: Multi-Stage Forwarding Chain");
$display("Expected: R8=100, R9=200, R10=300, R11=600\n");
uut.imem.memory[13] = {5'd5, 5'd0, 5'd8, 5'd0, 12'd100};       // ADDI R8, R0, 100
uut.imem.memory[14] = {5'd5, 5'd0, 5'd9, 5'd8, 12'd100};       // ADDI R9, R8, 100
uut.imem.memory[15] = {5'd5, 5'd0, 5'd10, 5'd9, 12'd100};      // ADDI R10, R9, 100
uut.imem.memory[16] = {5'd0, 5'd0, 5'd11, 5'd10, 5'd10, 7'd0}; // ADD R11, R10, R10

$display("TEST 14: Load-ALU-Store Chain");
$display("Expected: R12=1234, R13=1334, Mem[70]=1334\n");
uut.imem.memory[17] = {5'd10, 5'd0, 5'd12, 5'd0, 12'd100};     // LW R12, 100(R0)
uut.imem.memory[18] = {5'd5, 5'd0, 5'd13, 5'd12, 12'd100};     // ADDI R13, R12, 100
uut.imem.memory[19] = {5'd11, 5'd0, 5'd13, 5'd0, 12'd70};      // SW R13, 70(R0)

$display("TEST 15: JR with Register Dependency");
$display("Expected: R14=25, R15=888\n");
uut.imem.memory[20] = {5'd5, 5'd0, 5'd14, 5'd0, 12'd25};       // ADDI R14, R0, 25
uut.imem.memory[21] = {5'd14, 5'd0, 5'd0, 5'd14, 5'd0, 7'd0};  // JR R14
uut.imem.memory[22] = {5'd5, 5'd0, 5'd15, 5'd0, 12'd999};      // ADDI R15, R0, 999
uut.imem.memory[25] = {5'd5, 5'd0, 5'd15, 5'd0, 12'd888};      // ADDI R15, R0, 888
```

Figure 35: Test 4 Code part 2

- **Results**

```
# KERNEL: TEST 9: Load with Address Dependency
# KERNEL:   R23 = 30   (Expected: 30)    PASS
# KERNEL:   R24 = 555  (Expected: 555)   PASS
# KERNEL:
# KERNEL: TEST 10: Store with Dependencies
# KERNEL:   R22 = 42   (Expected: 42)    PASS
# KERNEL:   R25 = 40   (Expected: 40)    PASS
# KERNEL:   Mem[40] = 42   (Expected: 42)  PASS
# KERNEL:
# KERNEL: TEST 11: CALL and R31
# KERNEL:   R31 = 9    (Expected: 9)     PASS
# KERNEL:   R29 = 9    (Expected: 9)     PASS
# KERNEL:
# KERNEL: TEST 12: R31 Dependencies
# KERNEL:   R19 = 18   (Expected: 18)    PASS
# KERNEL:
# KERNEL: TEST 13: Multi-Stage Forwarding
# KERNEL:   R8  = 100  (Expected: 100)   PASS
# KERNEL:   R9  = 200  (Expected: 200)   PASS
# KERNEL:   R10 = 300  (Expected: 300)   PASS
# KERNEL:   R11 = 600  (Expected: 600)   PASS
# KERNEL:
# KERNEL: TEST 14: Load-ALU-Store Chain
# KERNEL:   R12 = 1234 (Expected: 1234)  PASS
# KERNEL:   R13 = 1334 (Expected: 1334)  PASS
# KERNEL:   Mem[70] = 1334 (Expected: 1334)  PASS
# KERNEL:
# KERNEL: TEST 15: JR with Dependency
# KERNEL:   R14 = 25   (Expected: 25)    PASS
# KERNEL:   R15 = 888  (Expected: 888)   PASS
# KERNEL:
```

Figure 36: Test 4 Result part 2

- **WaveForm**

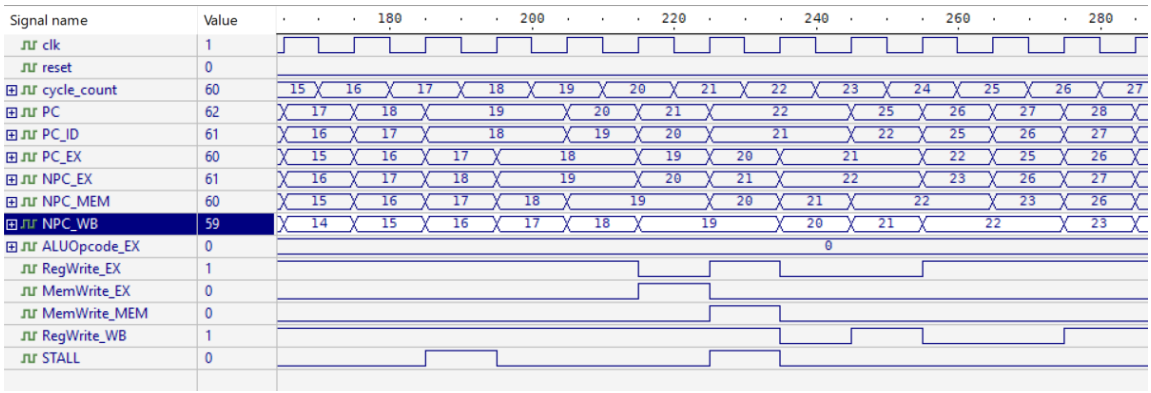


Figure 37: Test 4 waveform part 2

Discussion

In this set of tests, the processor goes through more situations where hazards are likely to appear in a pipelined RISC design. These tests aim to expose potential failure cases such as incorrect address generation, lost write-back values, or misuse of results that have not yet fully progressed through the pipeline.

Many of the tests focus on memory and address dependencies. The load with address dependency test checks that a load instruction waits for the correct address computed by a previous instruction, instead of accessing memory too early. The store with address dependency test makes sure that both the address and the data of a store instruction are valid when the memory write happens. The load ALU store chain combines all of this, showing that data can be loaded from memory, processed by the ALU, and then stored back correctly without writing incorrect values or causing unnecessary stalls.

Other tests focus on control flow and register dependencies. The multi-stage forwarding test confirms that results are forwarded across pipeline stages without waiting for write-back. The JR with dependency test also shows how it did update PC value even that it depends on a variable that updates in the WB stage in an instruction directly before it.

Overall, the waveform behavior and final register/memory contents showed through displayed statements confirm that the pipeline correctly integrates hazard detection, forwarding, stalling, and predicate control, ensuring correct execution under several tested scenarios.

Conclusion

In this project, a pipelined predicated RISC processor was designed and implemented using Verilog. The processor follows a five-stage pipeline (IF, ID, EX, MEM, WB) and supports arithmetic, memory, and control-flow instructions. Hazard detection, forwarding, stalling, flushing, and predicated execution were integrated to ensure correct and efficient operation. Simulation results confirm that the processor executes instructions correctly while handling data and control hazards effectively.

References

[1] :Instruction Set Architecture : https://www.arm.com/glossary/isa (accessed 29/12/2025)

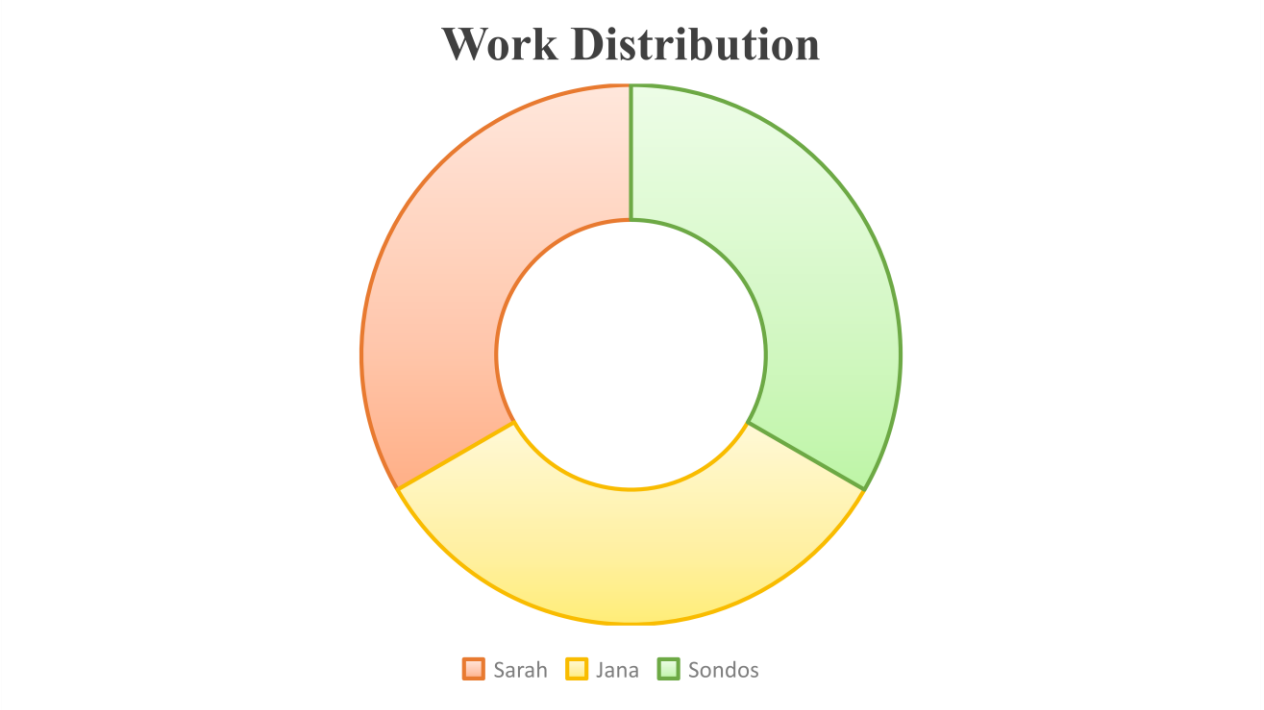
[2] :Arithmetic Instructions : http://www.cs.iit.edu/~virgil/cs470/Labs/Lab6.pdf (accessed 29/12/2025)

[3] Single cycle :https://www.geeksforgeeks.org/computer-organization-architecture/differences-between-single-cycle-and-multiple-cycle-datapath/

[4] Hazards https://www.geeksforgeeks.org/computer-organization-architecture/pipeline-hazards/

[5] https://byjus.com/gate/pipeline-hazards-in-computer-architecture-notes/

Teamwork



Our team worked closely throughout the entire project, starting from the processor design and datapath definition. We collaborated early on to establish a clear architecture that guided the implementation. Jana and Sondos mainly developed the source code for the core components, while Sarah focused on creating and validating the test benches. All team members also contributed to writing and integrating the main top-level module to ensure correct system operation.

During the Verilog implementation, tasks were divided while maintaining strong collaboration. Sondos implemented the ALU, register file and data memories. Sarah worked on the pipeline buffers and the connections between components, while Jana developed the hazard detection and forwarding units. The control unit and its signals were designed and verified jointly. For the report, Sondos handled the theory and datapath description, Jana wrote the code explanation and FSM, and Sarah documented the tests, verification results, and control signal truth tables.